

ACTSM - Assignment Block III

- Choose one of the following options for your assignment.
- Deadline: Monday, 29 June 2026, 23:59 [[aula global](#)]

Option A

1. Clone [essentia-plugins](#) repository and follow the [Quick Start](#) section in the README
 - a. Download and build dependencies (JUICE, Essentia, ...)
 - b. Install [Essentia prerequisites and dependencies](#)
 - i. Check if you need to install prerequisites
 - ii. eigen framework will be needed
 1. OSX: brew install eigen
 2. Linux: you have to [build it](#) or apt-get install
 - c. Build Essentia with waf
 - i. bash scripts/[build-essentia-osx.sh](#)
2. Create a new Essentia plugin based on PROJUCER
 - a. Choose an [Essentia algorithm](#) to implement in your plugin:
 - i. PeakDetection
 - ii. CrestFactor
 - iii. Audio2Pitch
 - iv. Median Filter
 - v. Moving Average
 - vi. DC Removal
 - vii. AllPass
 - viii. AutoCorrelation
 - ix. Clipper
 - x. Derivative
 - xi. Welch
 - xii. Danceability
 - xiii. Meter
 - xiv. Onset Detection
 - xv. Centroid
 - xvi. DistributionShape
 - xvii. Flatness
 - xviii. ChordsDetection
 - xix. Dynamic Complexity
 - xx. Intensity
 - xxi. Larm
 - xxii. Leq
 - xxiii. Loudness

xxiv. Loudness EBU128

1. Bear in mind that a descriptor, such as Loudness, would be an analyzer and a processor, such as Clipper, would be a processor.
 2. Check inputs and outputs for each algorithm.
- b. Duplicate the plugins/RMS_projucer folder
 - c. Rename the folder as <essentia-algorithm-name>_projucer
 - d. Open the RMS.jucer file in plugins/<essentia-algorithm-name>_projucer
 - e. Click in Project Settings button and rename the Project Name to <essentia-algorithm-name> (top left)
 - f. In the *Exporters* section, select your IDE and ensure that the eigen version and path in *Extra Compiler Flags* are available in your system.
 - i. # Show package information
 - ii. brew info eigen
 - iii. # Search for the eigen folder
 - iv. find / -name "Eigen" -type d 2>/dev/null

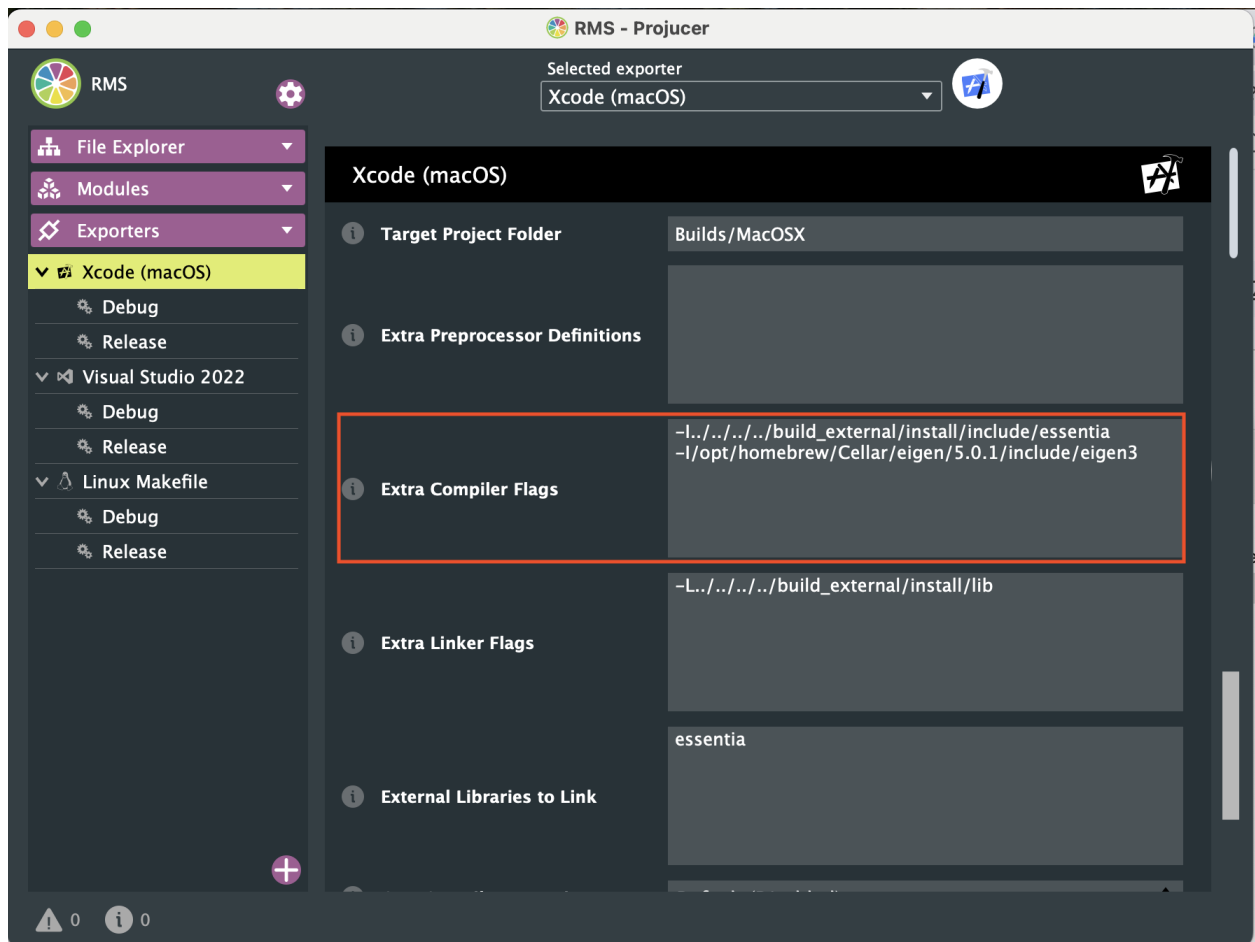


Figure 1. Projucer setup for linking essentia with extra flags

- g. To open the project click in the Save and Open in IDE button (top right)
- h. Build the VST3 output to confirm the project has been well set.

- i. You should see something similar to Figure 1 when you run `<essentia-algorithm-name>` plugin.

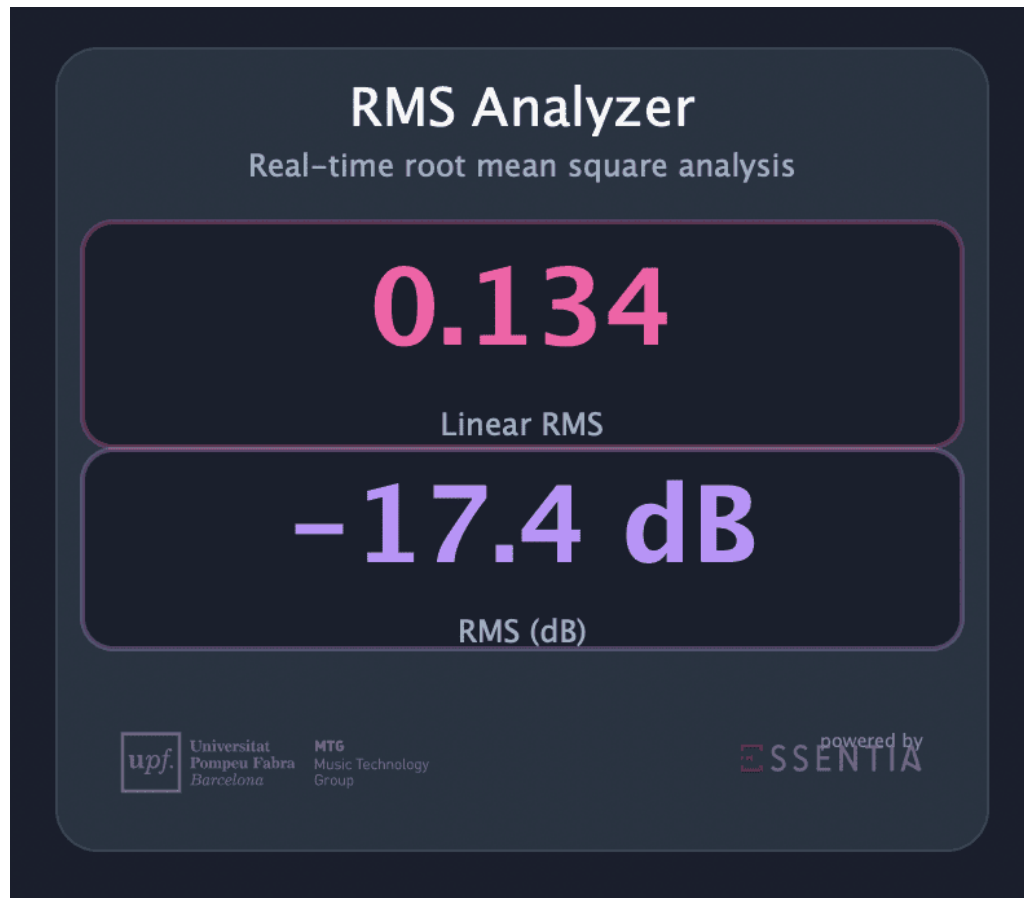


Figure 2. RMS plugin

- i. Replace all RMS references by `<essentia-algorithm-name>` in the class names and variables.
 - j. The plugin still should build and run properly.
3. Develop your plugin
- a. Replace the RMS algorithm in the header of the Plugin Processor by your algorithm.
 - b. Give support to your Essentia algorithm outputs.
 - c. If needed,
 - i. initialize additional buffers in `AudioProcessor::prepareToPlay()`
 - ii. update `AudioProcessor::releaseResources()`
 - d. In `AudioProcessor::prepareToPlay()` set the new outputs.
 - e. Update the `AudioProcessor::processBlock()`
 - f. In the editor, replace all RMS labels and references with the `<essentia-algorithm-name>`
 - g. Remove or replace decibel values if it doesn't make sense to your algorithm.

- h. Modify and tune your UI if needed.
 - i. Optional:
 - i. If your algorithm has parameters, implement buttons to control them. Check [Audio2Midi plugin](#) as reference.
- 4. Test it and contribute
 - a. Build the plugin and test in [REAPER](#) (or other DAW).
 - b. Make a short video as a demo.
 - c. If you have issues in the repository contact me or open an [issue](#).
 - d. Optional: make a PR into essentia-plugins to contribute
- 5. Deliver in Aula Global
 - a. Upload the source code in a zip file or provide a Github link.
 - b. Report all steps with comments, descriptions and screenshots.
 - c. Provide a short demo video.

Option B

- 1. Deliver all optional VAM assignments.
 - a. Notebooks are available in Aula Global
- 2. Deliver in Aula Global:
 - a. your edited notebooks with comments and results.
 - b. a report with comments, results, conclusions and figures.

Option C

Implement Panning Laws in a JUCE plugin

- 1. Create a processor plugin (audio2audio)
 - a. Set the Project Name as Panning
- 2. Implement the panning laws described [here](#)
 - a. Linear Panning
 - b. Constant Power Panning
 - c. -4.5 dB Pan Law
- 3. Process the input buffer with panning processing.
- 4. In the editor,
 - a. Create a slider to control the panning angle
 - b. Control the panning algorithm with a combo box
- 5. Delivery
 - a. Upload the source code in a zip file or provide a Github link.
 - b. Report all steps with comments, descriptions and screenshots.

- c. Provide a short demo video.

Option D

1. Please contact me via email if you choose this option.
2. You could use Blazej models or the pretrained reverb model introduced in the Neural Audio FX section in the VAM class for this assignment.
3. First, you will need to convert the model to ONNX
 - a. Use `torch.export.onnx()`
4. Setting the plugin project using the NeuroDrum Live plugin as reference.
 - a. Clone and build the NeuroDrum Live plugin
 - b. Copy it and rename the Projucer project
 - i. Check the External Libraries to Link in the exporter section (Figure 3)
 1. Click into the Xcode IDE to see them

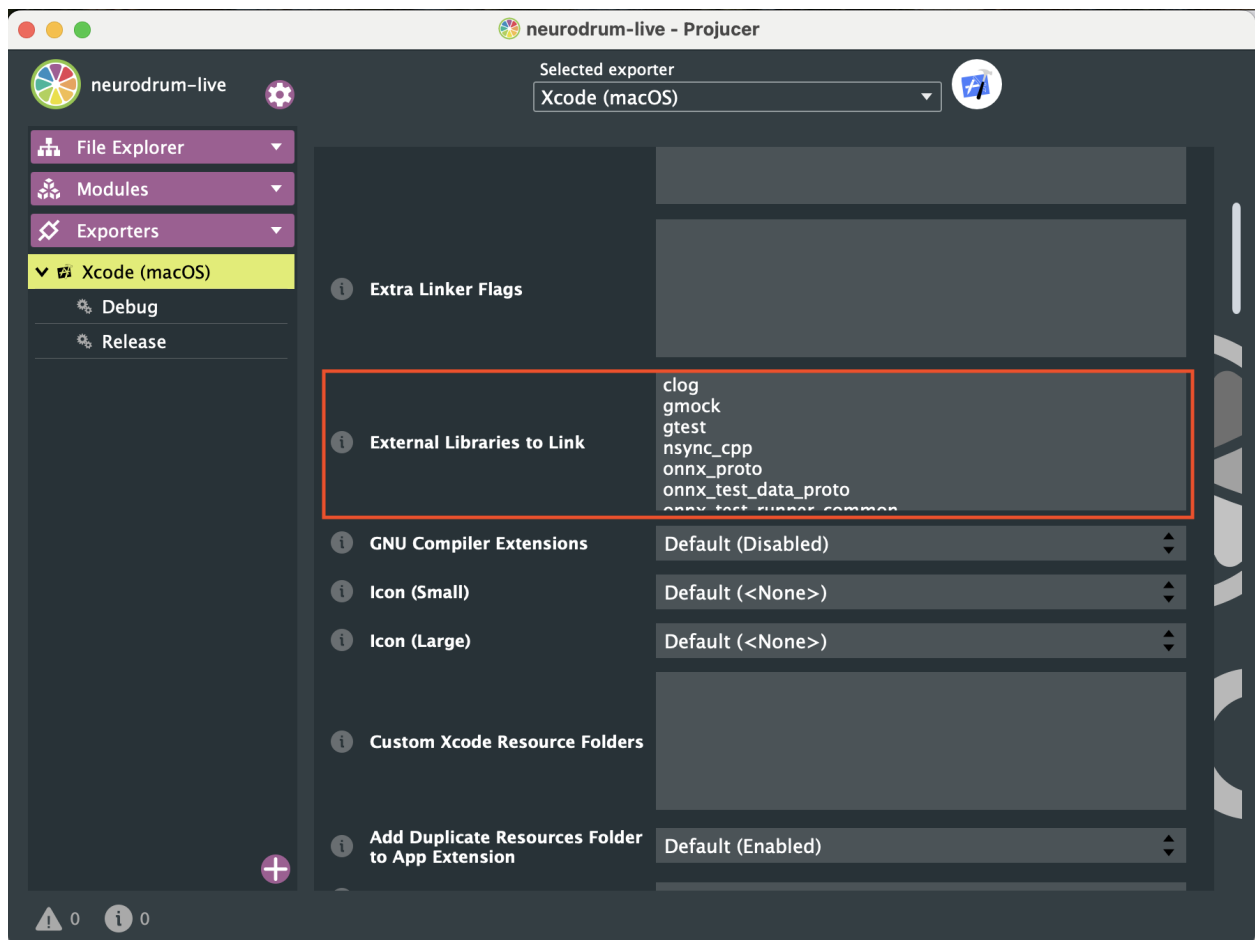


Figure 3. IDE setup for External Libraries

- ii. All those libraries should be built before and to locate them into the root folder in a lib directory as it defines the Extra Library Search Paths section (Figure 4).
 1. Some include paths should be also provided in the Header Search Paths section

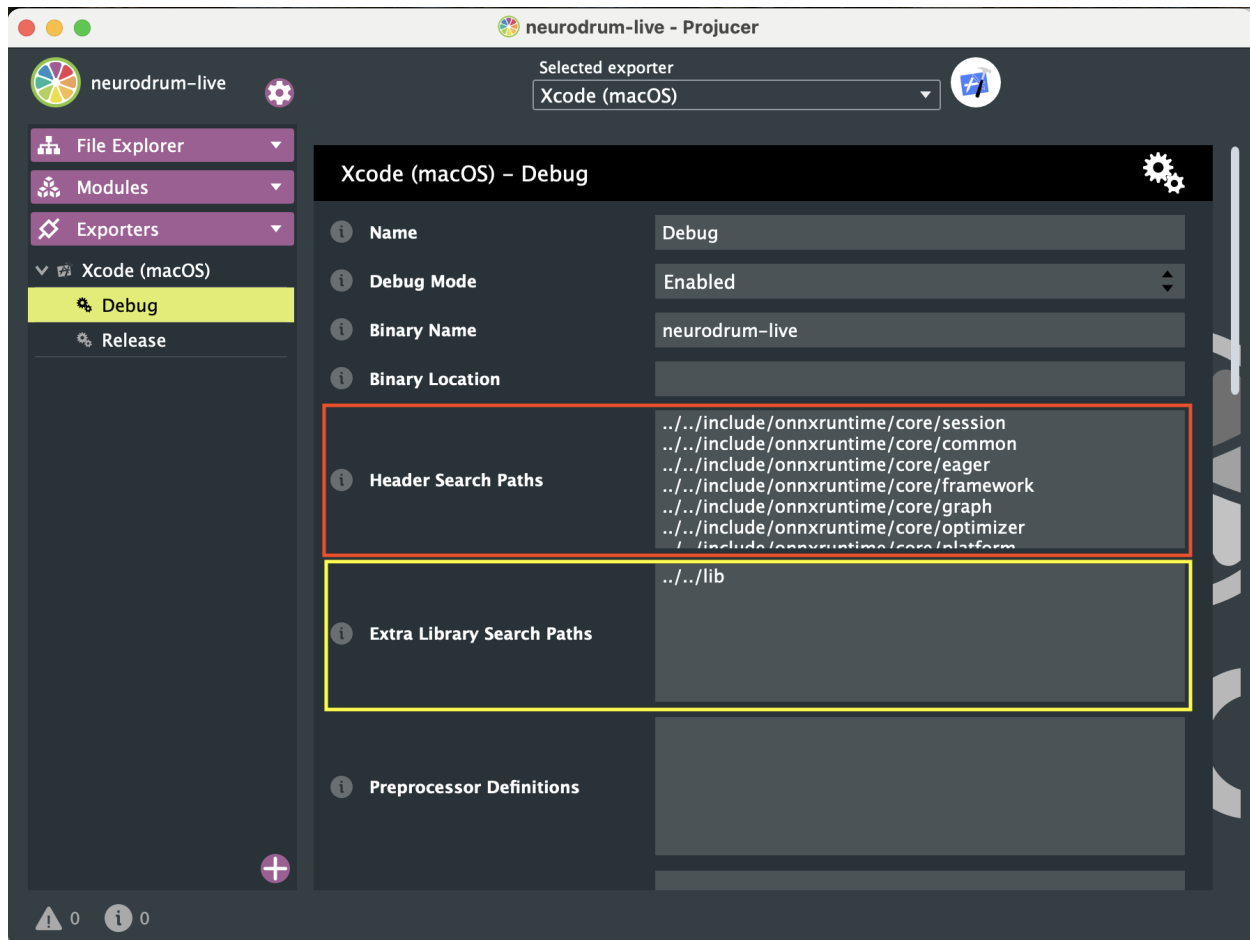


Figure 4: IDE setup for Debug mode for Header Paths and Extra Libraries

- iii. Check also the includes you will need to link with JUCE
 1. ../../include/onnxruntime/core/session
 2. ../../include/onnxruntime/core/common
 3. ../../include/onnxruntime/core/eager
 4. ../../include/onnxruntime/core/framework
 5. ../../include/onnxruntime/core/graph
 6. ../../include/onnxruntime/core/optimizer
 7. ../../include/onnxruntime/core/platform
 8. ../../include/onnxruntime/core/providers
- iv. All them should be build with ORT
- v. If this is too complex you could workaround with the [CMAKE version](#)
- c. Once the plugin works we have the setup ready to develop your plugin

5. Developing your AI plugin
 - a. Rename classes and variables if needed.
 - b. Modify the parameters and match them with your model parameters
 - c. Update model sizes
 - d. Update model input and sizes
 - e. Rethink and redo the UI
 - i. If it is a processor GENERATE button won't be needed.
6. Deliver in Aula Global
 - a. Upload the source code in a zip file or provide a Github link.
 - b. Report all steps with comments, descriptions and screenshots.
 - c. Provide a short demo video.

Some additional resources:

- Build onnxruntime: <https://onnxruntime.ai/docs/build/>
- ONNX example using the MNIST model for image classification:
 - https://github.com/microsoft/onnxruntime-inference-examples/blob/main/c_cxx/MNIST/MNIST.cpp
- ONNX Runtime example running a deep neural network model in JUCE (NeuroDrum)
 - Git: <https://github.com/Fyfe93/neurodrum-live/tree/main>
 - Model Inference is implemented [here](#)
 - MTG-Toolbox version [here](#)
 - NeuroDrum V2: <https://github.com/behzadhaki/neurodrum-live-v2>
- [Here](#) are the precompiled onnxruntime-libs
 - <https://github.com/csukuangfj/onnxruntime-libs/releases>
 - 1.22.0 releases [\[link\]](#)
- Building ONNX Runtime — Essentia script
 - https://github.com/xaviliz/essentia/blob/feat/onnx-predict/packaging/debian_3rdparty/build_onnx.sh